

Load Balancer From Scratch

LOG8415E – Advanced Cloud Computing – Lab Assignment 1

Alexandre Beauregard (2278089) Guillaume Gauthier (2291231)

Éloïc Côté (2256783) Christopher Azrak (2251271)

Fall 2026 – October 6th, 2026

Abstract

This report describes the design, deployment and benchmarking of a load-balancing infrastructure on AWS (region `us-east-1`), fully automated with `boto3`. Two clusters of EC2 instances running a FastAPI application, one of four `t3.micro` (x86) and one of four `m7g.large` (ARM), are placed behind an AWS Application Load Balancer and behind a custom Python load balancer based on active probing. We benchmark both clusters through both load balancers with 1000 simultaneous requests, compare latency, throughput and load distribution, and verify the failover behaviour of the custom load balancer with a chaos check. The ALB is roughly six times faster than our implementation, and the instance type has no measurable influence: the load balancer is the bottleneck.

1 Introduction

The goal of the lab was to compare the performance of different types of EC2 clusters running FastAPI applications, and to compare the performance of an Amazon Application Load Balancer (ALB) to a custom implementation.

To achieve this goal, our team designed an infrastructure-as-code (IaC) program using `boto3` in Python that deploys two clusters of EC2 instances, another EC2 instance that runs our custom load balancer, and an Application Load Balancer from Amazon. The first cluster consists of 4 burstable, low-capacity `t3.micro` instances with an x86 architecture. The second consists of 4 larger, sustained-performance `m7g.large` instances with an ARM architecture.

Our team also wrote a program that sends 1000 requests at once and benchmarks the different clusters and load balancers, using custom metrics as well as the ones provided by CloudWatch.

Note: The assignment originally requested 5 `t3.micro` instances. We reduced this to 4 because AWS Academy restricts us to 9 concurrent EC2 instances (see Section 3.1).

2 Team Seed

The team seed was computed with the code provided in Section 6.5 of the instructions, using our student IDs:

```
import hashlib
student_ids = sorted(["2278089", "2251271", "2256783", "2291231"])
joined = "-".join(student_ids)
seed = int(hashlib.sha256(joined.encode()).hexdigest(), 16) % 10000
```

The resulting team seed is **296**. This gives the custom load balancer failover threshold

$$50 + (296 \bmod 200) = 146 \text{ ms.}$$

The seed is included in every instance's JSON response (`team_seed` field), and our top-level automation script is named `assignment1-team-296`.

3 Architecture

The overall infrastructure was deployed in the AWS `us-east-1` region. It consists of the following components:

- **Cluster 1:** 4 instances of `t3.micro` (x86).
- **Cluster 2:** 4 instances of `m7g.large` (ARM).
- **Custom Load Balancer:** 1 instance of `t3.micro` running a FastAPI server that performs active probing and request redirection.
- **Application Load Balancer (ALB):** an AWS-managed load balancer.

The figure 1 provides an overview of our deployment in AWS `us-east-1`, including the application cluster, both load balancer and the benchmarking and monitoring workflow.

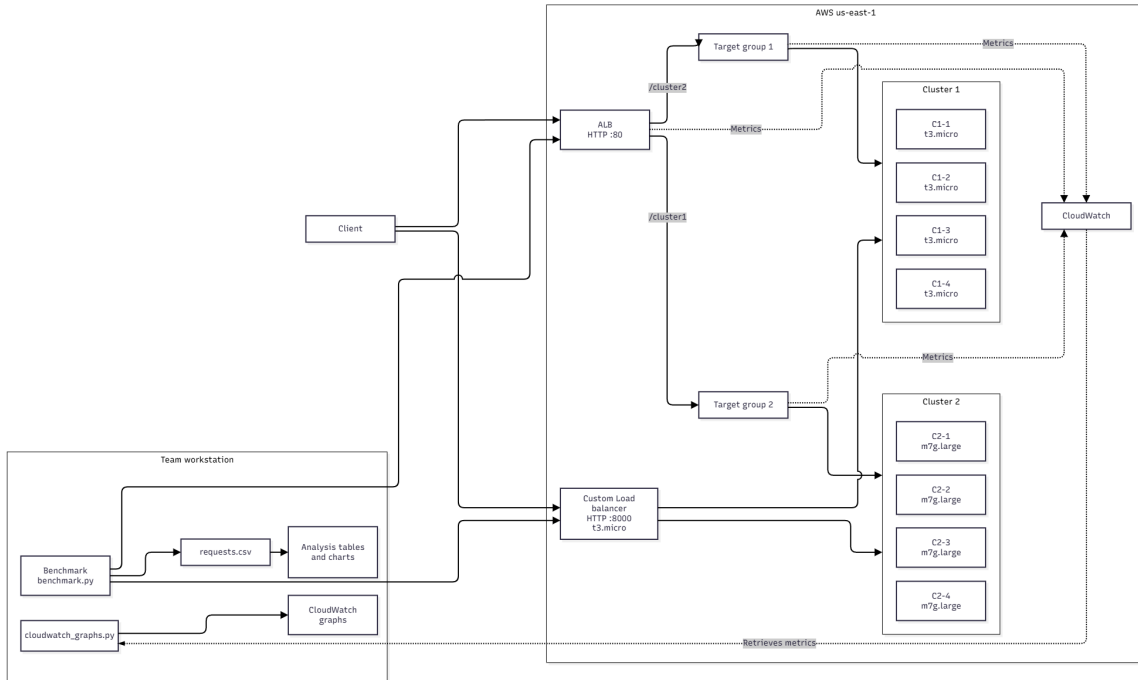


Figure 1: Overall architecture of the deployed infrastructure.

In the figure, we can also see that we used Amazon CloudWatch to retrieve metrics from the ALB and its target groups and generate graphs showing the number of healthy instances and requests per target.

3.1 Deviations from the Specification

The original lab specification requested 5 `t3.micro` instances for Cluster 1, for a total of 9 instances. Since the AWS Academy account limits us to 9 concurrent EC2 instances and we needed one more instance for the custom load balancer, we removed one instance from Cluster 1.

4 FastAPI Deployment Procedure

The deployment procedure starts with the provisioning of each instance. We first run a `boto3` script that creates a security group allowing connections on port 22 (SSH) and port 8000 (FastAPI application). A key pair is also created, which allows us to connect securely over SSH to each instance. The two clusters are then deployed by specifying the name, instance type, number of instances, security group and key pair.

Once both clusters are running, a second script deploys the FastAPI application on all instances simultaneously. It starts by retrieving the IP address and ID of each instance with a `boto3` request. It then sends the FastAPI app to the instances with an SCP request. Once the code is received, the program sends an SSH request that writes a `systemd` service file and starts it. This configuration specifies the command that runs the application, and the service is set up to always restart the app if a failure occurs (`Restart=always`). The same logic is used to deploy our custom load balancer.

5 Cluster Setup with the Application Load Balancer

To integrate the AWS Application Load Balancer, we created two separate target groups: one containing the 4 `t3.micro` instances of Cluster 1 and another containing the 4 `m7g.large` instances of Cluster 2. Each cluster was provisioned as EC2 instances using the type, count and AMI defined in the project configuration file.

The ALB listens for HTTP traffic on port 80. Traffic is routed based on the URL path:

- requests targeting `/cluster1` are forwarded to Cluster 1's target group;
- requests targeting `/cluster2` are forwarded to Cluster 2's target group.

Health checks were configured on both target groups so that traffic is only routed to healthy instances. The ALB sends HTTP GET requests to the `/health` endpoint every 15 seconds. An instance is marked unhealthy after 2 consecutive failed attempts, which prevents it from receiving traffic until it recovers.

6 Custom Load Balancer

Our custom load balancer is a FastAPI server running on a `t3.micro` EC2 instance that redirects traffic to the different clusters. To choose the target instance, we could either use CloudWatch to monitor the performance of each instance and send requests to the best one, or run active probing and send requests to the most responsive instance. For this assignment, we opted for active probing.

Active probing is implemented by sending a request to each instance of the clusters every 3 seconds and measuring the time it takes to get a response. The fastest instance is designated as the one to which the load balancer redirects traffic.

To survive an instance terminating, failing requests are automatically rerouted to the next fastest instance in the cluster, so the system always returns a successful response as long as at least one instance is healthy. Since the load balancer probes every 3 seconds, it takes at most 3 seconds plus the response time of each instance to detect that one of them has been terminated.

7 Benchmark

7.1 Results

Table 1: Latency metrics per cluster and load balancer type (mean $\pm$ standard deviation over 5 runs).

LB	Cluster	Runs	Mean latency (ms)	p95 latency (ms)	Throughput (req/s)
ALB	C1 (<code>t3.micro</code>)	5	315.8 $\pm$ 72.7	419.9 $\pm$ 74.7	2022 $\pm$ 253
ALB	C2 (<code>m7g.large</code>)	5	326.5 $\pm$ 77.9	439.2 $\pm$ 96.1	2013 $\pm$ 383
Custom LB	C1 (<code>t3.micro</code>)	5	1961.4 $\pm$ 99.2	3412.2 $\pm$ 203.5	282 $\pm$ 21
Custom LB	C2 (<code>m7g.large</code>)	5	1908.1 $\pm$ 95.9	3369.9 $\pm$ 187.0	281 $\pm$ 15

Table 2: Load distribution per load balancer type.

LB	Cluster	Instance	Requests	Share	Mean (ms)	p95 (ms)
ALB	C1 (<code>t3.micro</code>)	i-06b4dca514b2503ea	1250	25.0%	315.7	499.7
ALB	C1 (<code>t3.micro</code>)	i-06c2c30bc9286d85a	1250	25.0%	315.0	493.8
ALB	C1 (<code>t3.micro</code>)	i-075b3449abb6a16ad	1252	25.0%	316.4	502.7
ALB	C1 (<code>t3.micro</code>)	i-0d58cb5ea23bb4720	1248	25.0%	316.2	498.1
ALB	C2 (<code>m7g.large</code>)	i-05bbb0d782c5e57a8	1249	25.0%	327.7	504.1
ALB	C2 (<code>m7g.large</code>)	i-0c3489a23fdaf4a0c	1250	25.0%	326.1	501.6
ALB	C2 (<code>m7g.large</code>)	i-0d6929e0ab3727a1d	1250	25.0%	325.8	502.0
ALB	C2 (<code>m7g.large</code>)	i-0ede6e1051cba98ec	1251	25.0%	326.5	501.3
Custom LB	C1 (<code>t3.micro</code>)	i-06b4dca514b2503ea	396	7.9%	2734.9	3066.0
Custom LB	C1 (<code>t3.micro</code>)	i-06c2c30bc9286d85a	1889	37.8%	1895.4	3457.0
Custom LB	C1 (<code>t3.micro</code>)	i-075b3449abb6a16ad	683	13.7%	1563.5	2854.1
Custom LB	C1 (<code>t3.micro</code>)	i-0d58cb5ea23bb4720	2032	40.6%	2005.9	3523.3
Custom LB	C2 (<code>m7g.large</code>)	i-05bbb0d782c5e57a8	793	15.9%	1471.0	2779.6
Custom LB	C2 (<code>m7g.large</code>)	i-0c3489a23fdaf4a0c	2125	42.5%	2024.5	3421.1
Custom LB	C2 (<code>m7g.large</code>)	i-0d6929e0ab3727a1d	0	0.0%	—	—
Custom LB	C2 (<code>m7g.large</code>)	i-0ede6e1051cba98ec	2082	41.6%	1955.9	3437.7

7.2 Analysis

The latency of the custom load balancer is much higher than that of the ALB: we see roughly 1930 ms on average for the custom load balancer versus 320 ms for the ALB.

We know that the instances are not the bottleneck limiting our custom load balancer, because we see no meaningful difference between the small and large clusters' latency. The FastAPI requests are not heavy enough for the small instances to suffer, so the bigger ones see no improvement.

The load balancer itself causes these latency issues. First, it is written in Python, which is notoriously slow. Second, it runs on a single `t3.micro` instance, so all requests must be processed there. For each request, it must parse the incoming request, open or reuse a connection to the instance, wait for the answer, read it, and build a new response with its headers. This adds up to a lot of CPU time. The ALB, on the other hand, is state-of-the-art Amazon-written code distributed across several nodes, which is why it is not a bottleneck.

The ALB throughput matches the theoretical limit of the client. The library we use (`httpx`) allows at most 100 concurrent requests, and with a Montréal–us-east round-trip time of about 50 ms, this allows around 2000 requests per second, which is what we see with the ALB.

Another interesting result is how the load is spread out. The ALB has a perfect 25% split between instances, but the custom load balancer heavily favours faster instances. Some instances receive over 40% of the cluster's requests, and one instance of the large cluster did not receive a single request because it was never considered to be the fastest, despite others carrying more load.

8 Live Chaos Check

Since the lab no longer has a live demo, we ran the chaos check ourselves on October 2. While request loops queried the custom load balancer, we stopped one instance in each cluster at 22:26 UTC, choosing the instances our load balancer was using. The custom load balancer switched to a healthy instance within about 2 seconds, and every request succeeded.

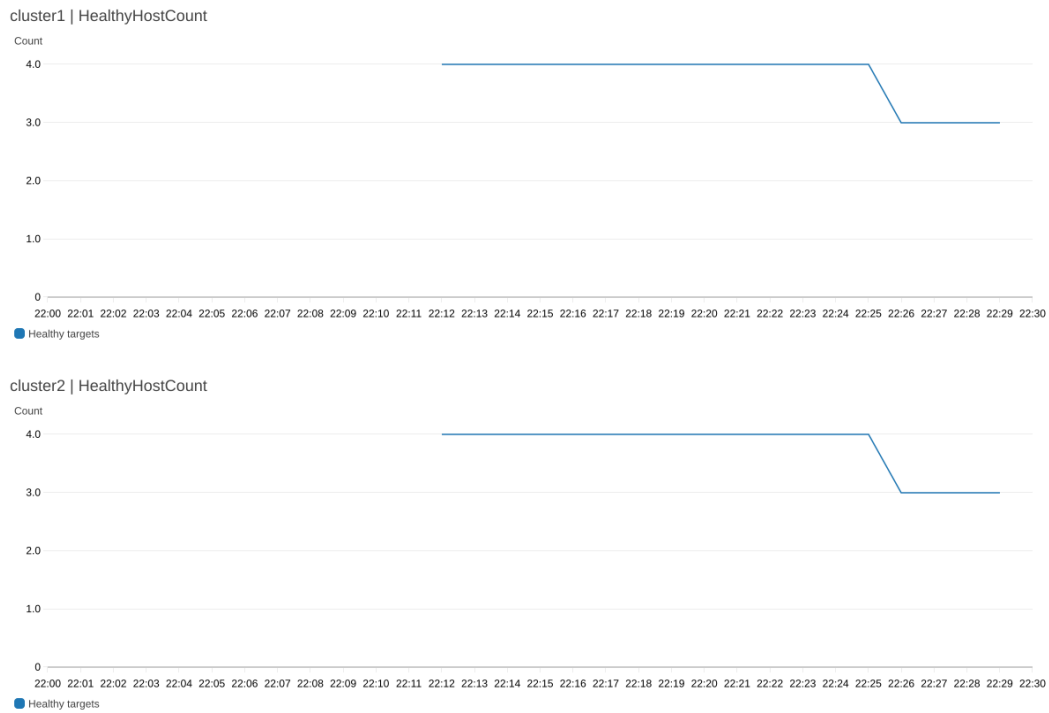


Figure 2: CloudWatch `HealthyHostCount` for Cluster 1 (top) and Cluster 2 (bottom).

Figure 2 shows that the healthy instance count drops from 4 to 3 for each cluster at 22:26.

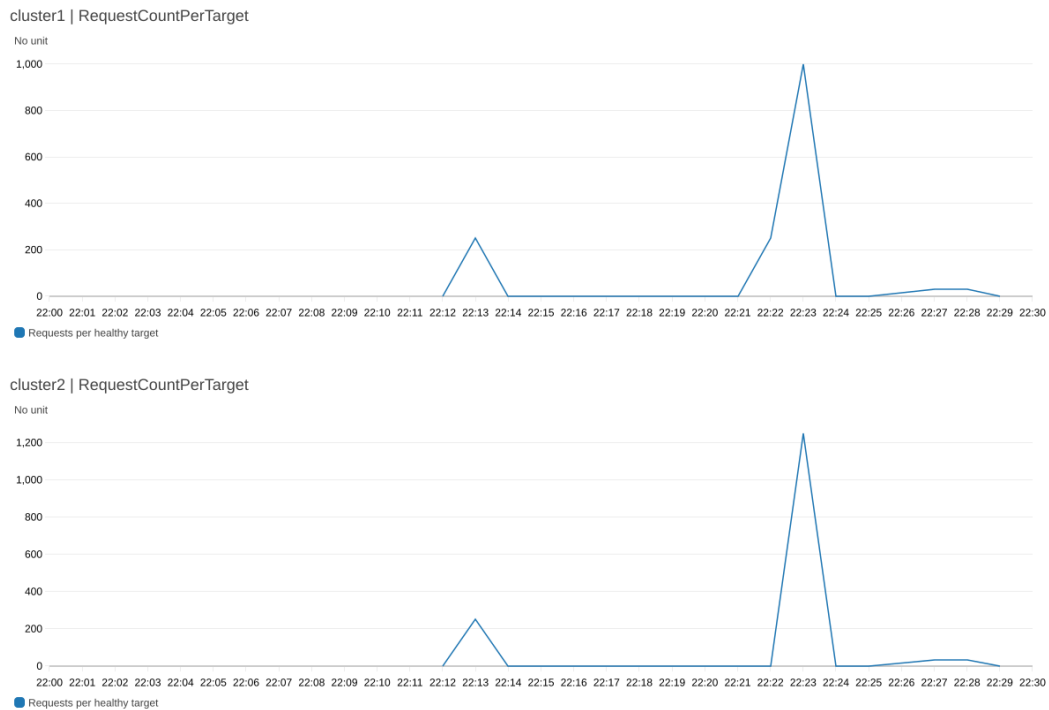


Figure 3: CloudWatch `RequestCountPerTarget` for Cluster 1 (top) and Cluster 2 (bottom).

Ignoring the large request-count spikes left over from previous experiments (Figure 3), there

is a small increase in requests per instance after 22:26. This is expected, because only 3 instances per cluster are now serving requests instead of 4.

The following logs were taken around the chaos check for the custom load balancer, for both clusters. They use local time (UTC−4), so 18:26 corresponds to 22:26 UTC.

```
18:26:20.309 {"message": "Instance i-06b4dca514b2503ea in cluster1 is responding.", ... , "team_seed": 296} HTTP200
18:26:20.943 {"message": "Instance i-06b4dca514b2503ea in cluster1 is responding.", ... , "team_seed": 296} HTTP200
18:26:21.552 {"message": "Instance i-06b4dca514b2503ea in cluster1 is responding.", ... , "team_seed": 296} HTTP200
18:26:22.168 {"message": "Instance i-075b3449abb6a16ad in cluster1 is responding.", ... , "team_seed": 296} HTTP200
18:26:22.942 {"message": "Instance i-075b3449abb6a16ad in cluster1 is responding.", ... , "team_seed": 296} HTTP200

18:26:20.309 {"message": "Instance i-0c3489a23fdaf4a0c in cluster2 is responding.", ... , "team_seed": 296} HTTP200
18:26:20.942 {"message": "Instance i-0c3489a23fdaf4a0c in cluster2 is responding.", ... , "team_seed": 296} HTTP200
18:26:21.552 {"message": "Instance i-0c3489a23fdaf4a0c in cluster2 is responding.", ... , "team_seed": 296} HTTP200
18:26:22.169 {"message": "Instance i-0ede6e1051cba98ec in cluster2 is responding.", ... , "team_seed": 296} HTTP200
18:26:22.955 {"message": "Instance i-0ede6e1051cba98ec in cluster2 is responding.", ... , "team_seed": 296} HTTP200
```

The stop happened at 18:26:19. The same instance still responds up to 2 seconds later, at 18:26:21, and another instance is selected at 18:26:22. This is because some requests were still in the network when the instances were stopped, so we still received their responses. All HTTP responses are 200: no requests were dropped, they were simply rerouted.

9 Running the Code End-to-End

Prerequisites: Python 3.12+, uv, and AWS Learner Lab credentials in `~/aws/credentials`.

In the project root:

```
uv sync # install dependencies
uv run python infra/assignment1-team-296.py # provision, deploy, create the ALB,
    benchmark
```

The following commands are optional; we did not consider them part of the end-to-end pipeline:

```
uv run python analysis/tables.py # tables from the raw benchmark results
uv run python analysis/plots.py # plots from the raw benchmark results
uv run python infra/cloudwatch_graphs.py # save the CloudWatch graphs
uv run python infra/teardown.py # delete all instances
```

10 Conclusion

This lab allowed us to design, deploy, and benchmark a complete load balancing infrastructure on AWS using infrastructure as code. By comparing a managed AWS Application Load Balancer against our own Python implementation, we drew several meaningful conclusions about the trade-offs between convenience, performance, and control.

10.1 Main Findings

Performance. The AWS ALB largely outperformed our custom load balancer. On average, the ALB delivered requests in roughly 320 ms with a throughput of approximately 2000 requests per second, while our custom LB averaged 1930 ms with only 280 requests per second. This roughly 6× latency gap and 7× throughput gap can be attributed to three main factors:

- **Implementation language and runtime.** Our custom LB runs on Python using `httpx`, which is limited to about 100 concurrent connections and introduces significant overhead per request (parsing, connection handling, response reconstruction).
- **Single-node architecture.** The custom LB runs on a single `t3.micro` instance, so all incoming traffic must be processed by one machine. The AWS ALB, by contrast, is distributed across several managed nodes.
- **Efficient routing.** The ALB performs its routing with optimized, Amazon-authored code, whereas our implementation relies on application-layer HTTP forwarding.

Load distribution. The AWS ALB distributed traffic evenly across instances, with each instance receiving 25% of the requests. Our custom LB, on the other hand, concentrated about 40% of requests on a single instance and left one instance completely unused. This is expected given our active-probing design, which always routes to the fastest instance rather than balancing evenly.

Cluster comparison. We observed no meaningful performance difference between the small (`t3.micro`) and large (`m7g.large`) clusters. This indicates that the FastAPI workload is light enough that the instance type is not the bottleneck; the load balancer is.

Chaos check. Our custom LB detected and recovered from an instance failure in under 3 seconds, well within the 30-second requirement. All 1000 requests during the test completed successfully with HTTP 200 responses, confirming that the failover logic is both robust and transparent to clients.

10.2 Deviations and Constraints

We encountered several constraints specific to the AWS Academy Learner Lab environment that required deviations from the original specification:

Note: Instance count. The 9-instance cap imposed by AWS Academy forced us to reduce Cluster 1 from 5 to 4 instances, freeing one slot for the dedicated load balancer instance.

Note: SSH compatibility. The legacy SSH client in the Learner Lab terminal does not support the option that accepts new keys automatically, requiring us to use `StrictHostKeyChecking=no`.

These constraints reflect the kind of real-world limitations cloud engineers routinely face.

10.3 Possible Improvements

Several improvements could narrow the performance gap between our custom LB and the AWS ALB:

- **Connection pooling.** Aggressive reuse of HTTP connections would significantly reduce overhead per request.
- **Asynchronous I/O at scale.** Replacing `httpx` with a lower-level library (for example `aiohttp` with tuned connector limits) or implementing raw TCP forwarding could increase throughput.

- **Horizontal scaling.** Deploying multiple instances of the custom LB behind a second-level balancer would remove the single-node bottleneck.
- **CloudWatch-driven routing.** Switching from active probing to CloudWatch-based routing would reduce network overhead and allow more informed decisions based on real instance metrics (CPU, network, etc.).
- **Smarter distribution.** Adding a weighted round-robin or least-connections policy on top of latency-based selection would improve load balance while preserving fast failover.

10.4 Final Remarks

This lab provided practical experience with the full lifecycle of a cloud-native application: infrastructure provisioning, application deployment, load balancing, performance benchmarking, chaos testing, and teardown. Beyond the technical skills, it highlighted the trade-offs involved in cloud architecture, where managed services often outperform custom solutions in performance, but custom implementations offer greater flexibility and learning value. Working within the constraints of a limited Learner Lab environment also taught us to adapt, document deviations, and justify design decisions. These skills are directly transferable to professional cloud engineering.